

BÁO CÁO MÔ TẢ CHI TIẾT QUÁ TRÌNH REAL-TIME

Hệ thống Phát hiện Gian lận Giao dịch theo Thời gian thực

(Real-time Fraud Detection Pipeline)

Thông tin	Giá trị
Tên hệ thống	Real-time Fraud Detection
Thư mục dự án	D:\ThucTap_VinSmartFuture\run_realtime
Ngày lập báo cáo	25/08/2026
Công nghệ chính	Apache Kafka 4.3.1, Apache Spark 4.0.1, PyTorch (LSTM + Attention), PostgreSQL 16, Docker Compose

Mục lục

- Tổng quan hệ thống
- Kiến trúc hệ thống
- Mô tả chi tiết từng giai đoạn của quá trình real-time
- Luồng dữ liệu và schema các Kafka topic
- Mô hình LSTM + Attention
- Nguyên tắc chống rò rỉ dữ liệu (No Data Leakage)
- Đo lường hiệu năng thời gian thực
- Quy trình vận hành
- Kết quả benchmark thực tế
- Mức tiêu thụ tài nguyên hệ thống
- Kết luận

1. Tổng quan hệ thống

1.1 Mục tiêu

Hệ thống mô phỏng và vận hành một **đường ống phát hiện gian lận giao dịch tài chính theo thời gian thực** với các chức năng chính:

- Nhận dòng giao dịch "mới" đến liên tục thông qua Kafka (broker trung gian).
- Tính toán **đặc trưng (features)** và **lịch sử giao dịch** của từng khách hàng / thiết bị theo thời gian thực.
- Sử dụng mô hình học sâu **LSTM + Attention** để dự đoán **xác suất gian lận** của từng giao dịch.
- Lưu toàn bộ kết quả dự đoán vào **PostgreSQL** phục vụ đánh giá độ chính xác và đo độ trễ (latency).

Vì không có nguồn giao dịch thật, hệ thống dùng bộ **dữ liệu giao dịch thẻ lịch sử** (từ 2018-10-01 đến 2020-05-22, lưu dưới dạng các file .pk1 ngày) và **"phát lại" (replay)** chúng với tốc độ cấu hình được (10 / 50 / 100 / 500 giao dịch mỗi giây) để mô phỏng dữ liệu đến theo thời gian thực.

1.2 Công nghệ sử dụng

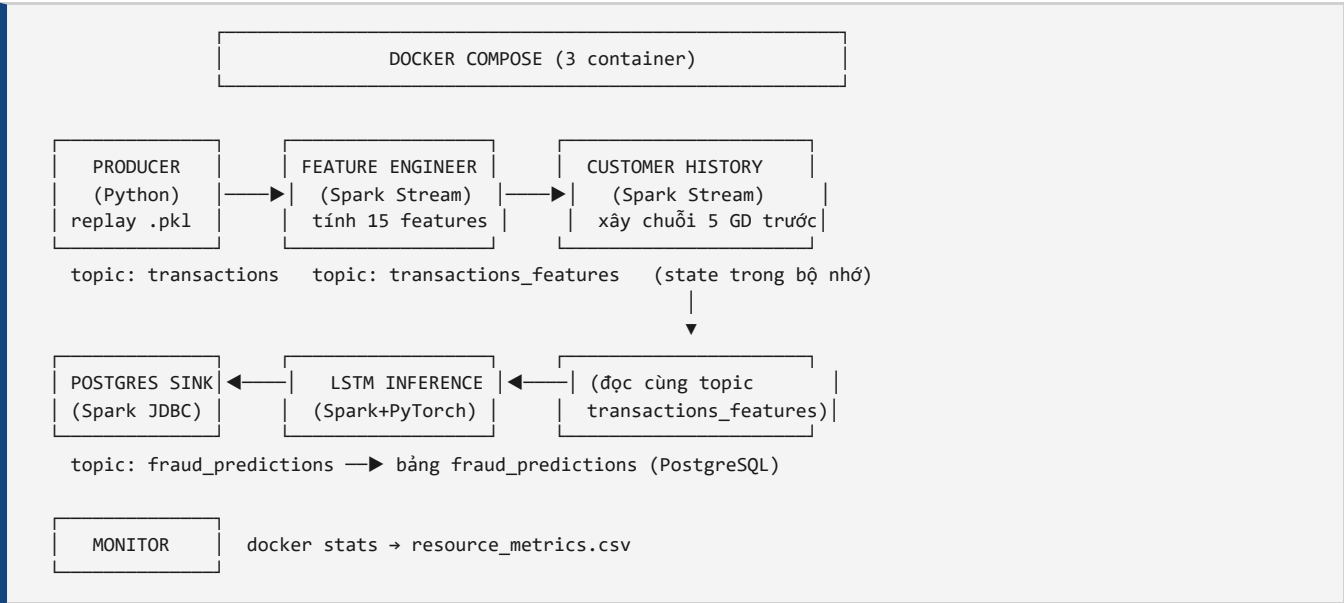
Thành phần	Công nghệ / Phiên bản
Message broker	Apache Kafka 4.3.1 (chế độ KRaft, 1 node)
Xử lý luồng dữ liệu	Apache Spark 4.0.1 – Structured Streaming
Mô hình dự đoán	PyTorch – LSTM + Attention (chạy trên CPU)
Cơ sở dữ liệu	PostgreSQL 16
Điều phối hạ tầng	Docker Compose (docker-compose.yml)
Producer / giám sát	Python (kafka-python, pandas, psycopg2, docker CLI)

1.3 Dữ liệu đầu vào

- ~**600 file** .pk1, mỗi file chứa toàn bộ giao dịch của một ngày (định dạng tên YYYY-MM-DD.pk1).
- Mỗi giao dịch gồm **8 cột**:

Cột	Ý nghĩa
TRANSACTION_ID	Mã giao dịch (khóa chính duy nhất)
TX_DATETIME	Thời điểm giao dịch trong bộ dữ liệu gốc
CUSTOMER_ID	Mã khách hàng
TERMINAL_ID	Mã thiết bị chấp nhận thẻ (terminal)
TX_AMOUNT	Số tiền giao dịch
TX_TIME_SECONDS	Thời gian trong ngày (giây)
TX_TIME_DAYS	Số ngày kể từ mốc thời gian
TX_FRAUD	Nhãn thật: 1 = gian lận, 0 = hợp lệ

1.4 Sơ đồ tổng thể pipeline



Pipeline gồm **6 giai đoạn chạy song song** trong các cửa sổ terminal riêng biệt (do `start_realtime.bat` khởi động):

1. **Kafka Producer** — phát lại giao dịch lịch sử vào Kafka.
2. **Feature Engineering** — tính 15 đặc trưng cho từng giao dịch.
3. **Customer History** — xây chuỗi 5 giao dịch trước của mỗi khách hàng (đầu vào chuỗi cho LSTM).
4. **LSTM Inference** — dự đoán xác suất gian lận bằng mô hình LSTM + Attention.
5. **PostgreSQL Sink** — ghi kết quả dự đoán xuống PostgreSQL.
6. **Resource Monitor** — theo dõi CPU/RAM của các container Docker.

1.5 Các Kafka topic

#	Topic	Giai đoạn ghi	Giai đoạn đọc
1	transactions	Producer	Feature Engineering
2	transactions_features	Feature Engineering	Customer History, LSTM Inference
3	fraud_predictions	LSTM Inference	PostgreSQL Sink

Ngoài các giao dịch bình thường, mỗi giai đoạn cuối của pipeline còn truyền đi một **END MARKER** (`__END_OF_STREAM__`) để báo hiệu dòng dữ liệu đã kết thúc; các giai đoạn hạ nguồn sẽ tự dừng có trật tự khi nhận được marker.

2. Kiến trúc hệ thống

2.1 Hạ tầng Docker Compose

File `docker-compose.yml` định nghĩa **3 container**:

Container	Image	Cổng	Vai trò
kafka	apache/kafka:4.3.1	9092 (EXTERNAL)	Message broker, chế độ KRaft (kết hợp broker + controller)
postgres	postgres:16	5432	Lưu trữ kết quả dự đoán
spark	realtime-fraud-spark:4.0.1 (dựng từ Dockerfile)	—	Chạy các Spark streaming job

Chi tiết cấu hình quan trọng:

- **Kafka:**
 - Listener nội bộ `INTERNAL://:29092` (dùng giữa các container, ví dụ Spark → `kafka:29092`).
 - Listener ngoài `EXTERNAL://:9092` (dùng cho Producer chạy trên máy chủ, `localhost:9092`).
 - Mỗi topic chỉ có **1 partition**, replication factor 1 (phù hợp môi trường thử nghiệm/đánh giá).
- **PostgreSQL:**
 - Database `fraud_detection`, user `fraud`, password `fraud123`.
 - Có healthcheck `pg_isready` để Spark chỉ khởi động sau khi PostgreSQL sẵn sàng.
 - Volume `postgres-data` giúp dữ liệu không bị mất khi container khởi động lại.

- **Spark:**
- Image dựng từ `apache/spark:4.0.1` + cài thêm `pandas`, `numpy`, `psycopg2-binary`, `torch`.
- Bind mount: `./spark` → `/opt/spark-apps` (mã nguồn Spark job), `./spark-data` → `/opt/spark-data` (checkpoint, features, history, logs), `./model` → `/opt/spark-models` (file checkpoint mô hình, mount chỉ đọc).
- `depends_on`: chờ Kafka khởi động và PostgreSQL healthy.

2.2 Cách các Spark job được khởi chạy

Mỗi giai đoạn Spark là một file Python được submit qua `spark-submit` bên trong container:

```
docker exec -it spark /opt/spark/bin/spark-submit \
  --conf spark.jars.ivy=/tmp/.ivy2 \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.1 \
  /opt/spark-apps/<ten_job>.py
```

- Giai đoạn **PostgreSQL Sink** cần thêm package `org.postgresql:postgresql:42.7.7`.
- Thư mục `/tmp/.ivy2` được dùng làm cache để tránh tải lại JAR mỗi lần chạy.

2.3 Checkpoint và trạng thái (state)

- Mỗi Spark job dùng **checkpoint riêng** dưới `/opt/spark-data/checkpoint/` để đảm bảo độ tin cậy và khôi phục sau khi khởi động lại:

Job	Đường dẫn checkpoint
Feature Engineering	<code>/opt/spark-data/checkpoint/feature_engineering</code>
Customer History	<code>/opt/spark-data/checkpoint/customer_history</code>
LSTM Inference	<code>/opt/spark-data/checkpoint/lstm_inference</code>
PostgreSQL Sink	<code>/opt/spark-data/checkpoint/postgres_sink</code>

- **Trạng thái rolling window** (lịch sử khách hàng, terminal) của Feature Engineering, Customer History và LSTM Inference được giữ **trong bộ nhớ của driver Python** (`defaultdict` + `deque`). Vì vậy checkpoint phải được xóa khi muốn chạy lại từ đầu (thực hiện trong `reset_realtime.bat`).

3. Mô tả chi tiết từng giai đoạn của quá trình real-time

3.1 Giai đoạn 1 — Kafka Producer (`producer/transaction_producer.py`)

Vai trò: mô phỏng nguồn giao dịch thời gian thực bằng cách phát lại dữ liệu lịch sử vào Kafka.

Luồng xử lý chi tiết:

1. **Đọc danh sách file .pk1** trong thư mục `data/` theo ngày từ `--start-date` đến `--end-date` (mặc định toàn bộ từ `2018-10-01` đến `2020-05-22`).
2. **Tải và chuẩn hóa từng file ngày** (`load_pk1_file`): - Chỉ giữ **8 cột bắt buộc** (`REQUIRED_COLUMNS`). - Chuyển `TX_DATETIME` sang dạng `datetime`; loại bỏ các dòng có ngày giờ không hợp lệ. - **Sắp xếp theo** `TX_DATETIME`, `TRANSACTION_ID` để đảm bảo thứ tự giao dịch đúng như dữ liệu gốc.

3. **Điều khiển tốc độ phát (rate control)** trong vòng lặp phát (`replay_once`): - `interval = 1.0 / rate` (ví dụ rate 10 tx/s → gửi 1 giao dịch mỗi 0.1 giây). - Dùng `time.perf_counter()` để đo thời gian và `time.sleep()` để canh nhịp, đảm bảo tốc độ thực tế khớp tốc độ cấu hình.
4. **Tạo thông điệp Kafka** (`row_to_message`): - Giữ nguyên 8 cột gốc. - **Thêm** `PRODUCER_TIMESTAMP` = thời gian thực (UTC, ISO-8601) tại thời điểm sắp gửi giao dịch. Đây chính là mốc thời gian để tính độ trễ end-to-end ở hạ nguồn. - `TX_FRAUD` **chỉ là ground truth** — không được dùng để tính đặc trưng hay dự đoán ở bất kỳ giai đoạn nào.
5. **Gửi tới Kafka**: - Bootstrap server `localhost:9092`, topic `transactions`, **partition 0**. - Cấu hình producer: `acks="all"`, `retries=5`, `linger_ms=2`, `batch_size=16384`, nén `gzip`; flush định kỳ mỗi 100 giao dịch. - In tiến độ mỗi `--print-every` giao dịch (hoặc mỗi rate giao dịch) kèm tốc độ thực tế.
6. **Hỗ trợ nhiều chế độ chạy** qua tham số dòng lệnh: - `--rate` (10/50/100/500), `--limit` (số giao dịch tối đa, ví dụ `500000`), `--print-every`, `--dry-run`, `--data-dir`, `--topic`, `--start-date`, `--end-date`. - Nếu chạy hết toàn bộ dữ liệu mà chưa đạt `--limit`, producer **tự động lặp lại từ đầu** để tiếp tục phát.
7. **Kết thúc**: - Khi đạt `--limit`, gửi **END MARKER** `__END_OF_STREAM__` (key `__END__`) vào topic để các giai đoạn hạ nguồn biết dòng dữ liệu đã xong và tự dừng.

Lệnh chạy điển hình:

```
cd /d D:\ThucTap_VinSmartFuture\run_realtime\producer
python transaction_producer.py --rate 100 --limit 500000 --print-every 1000
```

3.2 Giai đoạn 2 — Feature Engineering (`spark/feature_engineering.py`)

Vai trò: đọc giao dịch thô từ topic `transactions`, tính **15 đặc trưng** và ghi vào topic `transactions_features`.

Cách đọc dữ liệu:

- Dùng Spark Structured Streaming (`readStream` với `format("kafka")`, `subscribe = "transactions"`).
- Khi **chưa có checkpoint** → đọc từ `earliest` (toàn bộ dữ liệu trong topic); khi có checkpoint → đọc tiếp từ vị trí đã lưu.
- `maxOffsetsPerTrigger = 20000`: giới hạn số bản ghi tối đa mỗi micro-batch để tránh một batch quá lớn bị nạp hết vào bộ nhớ Python.
- Giải nén JSON value theo schema khai báo (`StructType`); bỏ qua các message không parse được.

Cách xử lý trong mỗi micro-batch (`foreachBatch`):

1. Sắp xếp các giao dịch theo `TX_DATETIME` (giữ nguyên logic lịch sử của notebook huấn luyện).
2. Với mỗi giao dịch, **kiểm tra tính hợp lệ** (bỏ qua nếu `TRANSACTION_ID`, `TX_DATETIME`, `CUSTOMER_ID`, `TERMINAL_ID` bị NULL; mặc định `TX_AMOUNT = 0`, `TX_FRAUD = 0`).
3. **Tính 15 đặc trưng** (danh sách chi tiết trong Mục 4.2).

15 đặc trưng:

#	Feature	Nhóm	Công thức
1	TX_AMOUNT	Giao dịch	Số tiền giao dịch
2	TX_DURING_WEEKEND	Giao dịch	1 nếu giao dịch vào Thứ 7 / Chủ nhật (dayofweek)
3	TX_DURING_NIGHT	Giao dịch	1 nếu giờ giao dịch < 6 (hour)
4	CUSTOMER_ID_NB_TX_1DAY_WINDOW	Khách hàng	Số giao dịch của khách trong 1 ngày gần nhất
5	CUSTOMER_ID_AVG_AMOUNT_1DAY_WINDOW	Khách hàng	Số tiền trung bình/giao dịch trong 1 ngày
6	CUSTOMER_ID_NB_TX_7DAY_WINDOW	Khách hàng	Số giao dịch trong 7 ngày
7	CUSTOMER_ID_AVG_AMOUNT_7DAY_WINDOW	Khách hàng	Số tiền trung bình trong 7 ngày
8	CUSTOMER_ID_NB_TX_30DAY_WINDOW	Khách hàng	Số giao dịch trong 30 ngày
9	CUSTOMER_ID_AVG_AMOUNT_30DAY_WINDOW	Khách hàng	Số tiền trung bình trong 30 ngày
10	TERMINAL_ID_NB_TX_1DAY_WINDOW	Terminal	Số giao dịch tại terminal trong 1 ngày
11	TERMINAL_ID_RISK_1DAY_WINDOW	Terminal	Tỷ lệ gian lận tại terminal trong 1 ngày
12	TERMINAL_ID_NB_TX_7DAY_WINDOW	Terminal	Số giao dịch tại terminal trong 7 ngày
13	TERMINAL_ID_RISK_7DAY_WINDOW	Terminal	Tỷ lệ gian lận trong 7 ngày
14	TERMINAL_ID_NB_TX_30DAY_WINDOW	Terminal	Số giao dịch tại terminal trong 30 ngày
15	TERMINAL_ID_RISK_30DAY_WINDOW	Terminal	Tỷ lệ gian lận trong 30 ngày

Cơ chế tính window (quan trọng, chống rò rỉ):

- State là `customer_state` và `terminal_state` (`defaultdict + deque`), lưu `(timestamp, amount | fraud)` của các giao dịch đã qua; các bản ghi cũ hơn 30 ngày bị loại bỏ.
- **Customer features:** tính từ lịch sử các giao dịch **trước đó** của khách, sau đó cộng +1 số lần và gộp `TX_AMOUNT` hiện tại vào trung bình — để **khớp đúng rolling window của notebook huấn luyện** (vốn bao gồm cả giao dịch hiện tại).
- **Terminal features:** tính từ lịch sử trước đó (không gồm giao dịch hiện tại).
- Giao dịch hiện tại chỉ được **chèn vào state sau khi** các đặc trưng của nó đã được tính xong (chống target/current-row leakage).

Ghi kết quả:

- Tạo DataFrame `output_df` với schema cố định, kiểm tra số dòng/ID trùng lặp trong batch, sau đó ghi vào Kafka topic `transactions_features` dạng JSON, **key** = `TRANSACTION_ID`.
- Khi nhận được END MARKER từ topic đầu vào, ghi tiếp marker xuống `transactions_features` rồi dừng ứng dụng.
- Checkpoint tại `/opt/spark-data/checkpoint/feature_engineering`.

3.3 Giai đoạn 3 — Customer History (`spark/customer_history.py`)

Vai trò: xây dựng **chuỗi (sequence) 5 giao dịch gần nhất** của từng khách hàng — đúng dạng đầu vào mà mô hình LSTM cần, đồng thời kiểm chứng cấu trúc `x / y` trước khi dự đoán.

Cách hoạt động:

1. Đọc topic `transactions_features` bằng Spark Structured Streaming.

2. Duy trì state `customer_history` (`defaultdict + deque`) lưu tối đa `SEQ_LEN = 5` giao dịch gần nhất của mỗi `CUSTOMER_ID`.
3. Với mỗi giao dịch hiện tại (gọi là T6): - `x` = vector đặc trưng của **5 giao dịch trước đó** T1 → T5 (mỗi giao dịch là 1 vector 15 đặc trưng) → shape `(5, 15)`. - `y` = nhãn `TX_FRAUD` của giao dịch hiện tại. - **Giao dịch hiện tại KHÔNG được chèn vào `x`** (tránh target/current-row leakage).
4. Sau khi xây xong `x`, giao dịch hiện tại mới được nối vào lịch sử và cắt còn 5 phần tử gần nhất.
5. Kiểm tra cảnh báo nếu toàn bộ giá trị của `x` bằng 0 (giúp phát hiện lỗi ở Feature Engineering hoặc Kafka).

Lưu ý: giai đoạn này **không ghi ra Kafka** — nó duy trì state ngay trong bộ nhớ driver và dùng để kiểm chứng/warm-up. Giai đoạn LSTM Inference độc lập đọc lại topic `transactions_features` và tự xây lại chuỗi tương tự.

- Checkpoint: `/opt/spark-data/checkpoint/customer_history`.
- Dừng khi nhận END MARKER.

3.4 Giai đoạn 4 — LSTM Inference (`spark/lstm_inference.py`)

Vai trò: tải mô hình LSTM + Attention đã huấn luyện, dự đoán **xác suất gian lận** cho từng giao dịch và ghi kết quả vào topic `fraud_predictions`.

Cách hoạt động chi tiết:

1. **Đọc dữ liệu:** Spark Structured Streaming đọc topic `transactions_features`.
2. **Xây chuỗi:** duy trì lịch sử 5 giao dịch gần nhất của mỗi khách hàng trong bộ nhớ. Với mỗi giao dịch hiện tại: - `x` = chuỗi 5 giao dịch trước, mỗi giao dịch 15 đặc trưng. - Khi khách hàng chưa có đủ 5 giao dịch, chuỗi được **left zero-padding** (chèn các vector 0 vào đầu) để vẫn tạo được dự đoán cho **mọi** giao dịch:
 - T1 → `x` = `[0,0,0,0,0]`
 - T2 → `x` = `[0,0,0,0,T1]`
 - T3 → `x` = `[0,0,0,T1,T2]`
 - T4 → `x` = `[0,0,T1,T2,T3]`
 - T5 → `x` = `[0,T1,T2,T3,T4]`
 - T6 → `x` = `[T1,T2,T3,T4,T5]`
3. **Chuẩn hóa đặc trưng:** các đặc trưng real-time là dạng thô (RAW). Mô hình được huấn luyện trên dữ liệu đã chuẩn hóa bằng `StandardScaler` (fit trên cửa sổ huấn luyện 2018-07-11 → 2018-07-18 trong notebook `Tham_DuBao_GianLan_GiaoDich.ipynb`). Do đó mỗi giá trị được chuẩn hóa trước khi đưa vào mô hình bằng **đúng mean/std của tập huấn luyện** (các hằng số `FEATURE_MEAN` / `FEATURE_STD` được nhúng sẵn trong file).
4. **Dự đoán theo batch (BATCHED INFERENCE):** - Gộp tất cả mẫu `x` của batch thành tensor `(N, 5, 15)`. - Ghi nhận `PREDICTION_START_TIMESTAMP` (UTC) và `start = perf_counter()`. - Gọi `model(x_batch)` một lần trong `torch.no_grad()`. - Ghi nhận `PREDICTION_END_TIMESTAMP` và `end = perf_counter()`. - `batch_latency_ms = (end - start) * 1000` — đây là **prediction latency** của cả batch (mỗi giao dịch trong batch nhận cùng giá trị này).
5. **Tính kết quả:** - `probability` = đầu ra sigmoid của mô hình (float 0..1). - `FRAUD_PREDICTION = 1` nếu `probability >= THRESHOLD`, ngược lại 0. - Ghi nhận `END_TO_END_LATENCY_MS` (tạm tính tại đây) = `PREDICTION_END_TIMESTAMP - PRODUCER_TIMESTAMP`. - Lưu các thống kê latency (current batch + cumulative).

6. **Ghi Kafka:** tạo DataFrame `output_df` với schema cố định gồm các cột: `TRANSACTION_ID`, `TX_DATETIME`, `PRODUCER_TIMESTAMP`, `CUSTOMER_ID`, `TERMINAL_ID`, `TX_AMOUNT`, `FRAUD_PROBABILITY`, `FRAUD_PREDICTION`, `THRESHOLD`, `TX_FRAUD`, `PREDICTION_START_TIMESTAMP`, `PREDICTION_END_TIMESTAMP`, `PREDICTION_LATENCY_MS`, `END_TO_END_LATENCY_MS`. Ghi JSON vào topic `fraud_predictions`, **key** = `TRANSACTION_ID`.

Ghi chú về ngưỡng (THRESHOLD): mã nguồn hiện tại của `lstm_inference.py` cấu hình `THRESHOLD = 0.7`; tuy nhiên các lần benchmark gần đây (log `lstm_inference.log`, dữ liệu trong PostgreSQL) ghi nhận ngưỡng **0.5** — tương đương cấu hình trong `model_inference.py` và `test_lstm.py`.

- Checkpoint: `/opt/spark-data/checkpoint/lstm_inference`.
- Mô hình: `/opt/spark-models/Tham_lstm_attn_checkpoint.pth`.

3.5 Giai đoạn 5 — PostgreSQL Sink (`spark/postgres_sink.py`)

Vai trò: đọc kết quả dự đoán từ topic `fraud_predictions` và **ghi xuống bảng** `fraud_predictions` trong PostgreSQL — đây là nơi chứa dữ liệu cuối cùng để đánh giá.

Cách hoạt động chi tiết:

1. Spark Structured Streaming đọc topic `fraud_predictions`, parse JSON theo schema.
2. Chuyển đổi các trường timestamp sang kiểu dữ liệu tương ứng.
3. **Tạo** `SINK_TIMESTAMP` = `current_timestamp()` tại thời điểm xử lý batch (mốc thời gian "đầu ra cuối cùng" của pipeline).
4. **Tính lại** `END_TO_END_LATENCY_MS` chính xác tại điểm cuối: `END_TO_END_LATENCY_MS` = `unix_micros(SINK_TIMESTAMP) - unix_micros(PRODUCER_TIMESTAMP)` - Dùng `unix_micros()` để giữ độ chính xác đến micro-giây. - **Không dùng** `TX_DATETIME` (thời điểm giao dịch trong bộ dữ liệu gốc) cho latency real-time.
5. Sắp xếp theo `TX_DATETIME`, gộp về 1 partition (`coalesce(1)`) để các dòng được ghi **tuần tự** qua một kết nối JDBC — giữ nguyên thứ tự trong PostgreSQL.
6. Ghi qua JDBC: - URL `jdbc:postgresql://postgres:5432/fraud_detection`, bảng `fraud_predictions`. - User/password `fraud / fraud123`, driver `org.postgresql.Driver`. - `batchsize=1000`, `stringtype=unspecified`, mode `append`.
7. Dừng khi nhận END MARKER; checkpoint tại `/opt/spark-data/checkpoint/postgres_sink`.

Bảng `fraud_predictions` (schema):

Cột	Kiểu	Ý nghĩa
TRANSACTION_ID	int	Mã giao dịch (khóa chính)
TX_DATETIME	timestamp	Thời điểm giao dịch gốc
PRODUCER_TIMESTAMP	timestamp	Thời điểm producer gửi
CUSTOMER_ID / TERMINAL_ID	int	Khách hàng / terminal
TX_AMOUNT	double	Số tiền
FRAUD_PROBABILITY	double	Xác suất gian lận mô hình trả ra
FRAUD_PREDICTION	int	Nhãn dự đoán (0/1)
THRESHOLD	double	Ngưỡng đã dùng
TX_FRAUD	int	Nhãn thật (ground truth)
PREDICTION_START/END_TIMESTAMP	timestamp	Mốc bắt đầu/kết thúc inference
PREDICTION_LATENCY_MS	double	Độ trễ dự đoán
END_TO_END_LATENCY_MS	double	Độ trễ từ producer đến sink
SINK_TIMESTAMP	timestamp	Thời điểm ghi vào PostgreSQL

3.6 Giai đoạn 6 — Resource Monitor (`evaluate/monitor_resources.py`)

Vai trò: theo dõi tài nguyên CPU/RAM của các container Docker trong suốt thời gian chạy benchmark.

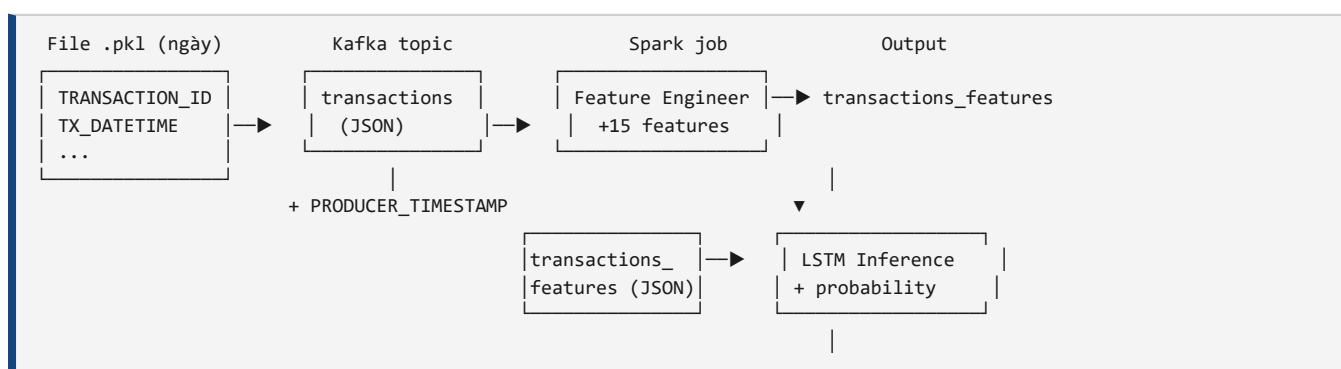
Cách hoạt động:

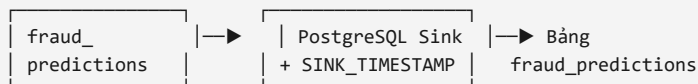
- Chạy lệnh `docker stats --no-stream` định kỳ (mặc định mỗi 1 giây) trong suốt `--duration` giây.
- Ghi ra file CSV `resource_metrics.csv` với các cột: `timestamp`, `container`, `cpu_percent`, `memory_usage`, `memory_percent`.
- Dùng để đánh giá gánh nặng hệ thống theo từng tốc độ giao dịch.

```
cd /d D:\ThucTap_VinSmartFuture\run_realtime\evaluate
python monitor_resources.py --duration 300 --interval 1
```

4. Luồng dữ liệu và schema các Kafka topic

4.1 Hành trình của một giao dịch qua pipeline





1. Producer đọc từng dòng giao dịch từ file `.pk1`, thêm `PRODUCER_TIMESTAMP` rồi gửi **JSON** vào topic `transactions`.
2. Feature Engineering đọc `transactions`, tính 15 đặc trưng và ghi JSON vào `transactions_features` (giữ `TRANSACTION_ID`, `PRODUCER_TIMESTAMP`, `TX_DATETIME`, `TX_FRAUD`).
3. LSTM Inference đọc `transactions_features`, tự xây chuỗi 5 giao dịch trước của khách, chuẩn hóa, dự đoán và ghi JSON vào `fraud_predictions`.
4. PostgreSQL Sink đọc `fraud_predictions`, thêm `SINK_TIMESTAMP`, tính `END_TO_END_LATENCY_MS` và ghi vào bảng `fraud_predictions` trong PostgreSQL.

4.2 Schema từng topic

Topic `transactions` (do Producer ghi):

Cột	Kiểu (JSON)
<code>TRANSACTION_ID</code>	integer
<code>TX_DATETIME</code>	string (ISO-8601)
<code>CUSTOMER_ID</code>	integer
<code>TERMINAL_ID</code>	integer
<code>TX_AMOUNT</code>	number
<code>TX_TIME_SECONDS</code>	number
<code>TX_TIME_DAYS</code>	number
<code>TX_FRAUD</code>	integer (0/1 — ground truth)
<code>PRODUCER_TIMESTAMP</code>	string (ISO-8601 UTC — thời gian thực)

Topic `transactions_features` (do Feature Engineering ghi):

Cột	Kiểu
<code>TRANSACTION_ID</code> , <code>PRODUCER_TIMESTAMP</code> , <code>TX_DATETIME</code> , <code>CUSTOMER_ID</code> , <code>TERMINAL_ID</code> , <code>TX_AMOUNT</code> , <code>TX_FRAUD</code>	như trên
<code>TX_DURING_WEEKEND</code> , <code>TX_DURING_NIGHT</code>	integer (0/1)
6 cột khách hàng: <code>CUSTOMER_ID_NB_TX_{1,7,30}DAY_WINDOW</code> , <code>CUSTOMER_ID_AVG_AMOUNT_{1,7,30}DAY_WINDOW</code>	number
6 cột terminal: <code>TERMINAL_ID_NB_TX_{1,7,30}DAY_WINDOW</code> , <code>TERMINAL_ID_RISK_{1,7,30}DAY_WINDOW</code>	number

Tổng cộng **15 đặc trưng** + các trường định danh/huyết thống (ID, timestamp, ground truth).

Topic `fraud_predictions` (do LSTM Inference ghi):

Cột	Kiểu
TRANSACTION_ID, TX_DATETIME, PRODUCER_TIMESTAMP, CUSTOMER_ID, TERMINAL_ID, TX_AMOUNT, TX_FRAUD	như trên
FRAUD_PROBABILITY	number (0..1)
FRAUD_PREDICTION	integer (0/1)
THRESHOLD	number
PREDICTION_START_TIMESTAMP, PREDICTION_END_TIMESTAMP	string (ISO-8601 UTC)
PREDICTION_LATENCY_MS	number
END_TO_END_LATENCY_MS	number

5. Mô hình LSTM + Attention

5.1 Kiến trúc

Mô hình dự đoán gian lận được huấn luyện offline (notebook `Tham_DuBao_GianLan_GiaoDich.ipynb`) và chỉ **tải trọng số (checkpoint)** để suy diễn trong thời gian thực. Kiến trúc gồm 2 lớp chính:

- LSTM (nn.LSTM):** - Input: (batch, seq_len=5, num_features=15). - Số hidden units: 100, 1 lớp, không dropout. - Đầu ra: chuỗi các hidden state (batch, 5, 100).
- Attention (cơ chế chú ý kiểu Bahdanau đơn giản):** - $attn = output \otimes context^T$ (batch matrix multiply) → softmax theo từng vị trí. - $mix = attn \otimes context$ (weighted sum của các hidden state). - $combined = concat(mix, output) \rightarrow$ qua linear projection ($100*2 \rightarrow 100$) → tanh.
- Đầu ra dự đoán:** - Lấy hidden state cuối (sau attention), qua linear layer rồi sigmoid → **1 giá trị xác suất (0..1)**.

```

Input (1, 5, 15)
  |
  v
LSTM(hidden=100) → Hidden states (1, 5, 100)
  |
  v
Attention: attn = softmax(score) ; mix = attn ⊗ context ; concat → tanh(linear)
  |
  v
Linear + Sigmoid → FRAUD_PROBABILITY (0..1)

```

5.2 Tham số suy diễn

Tham số	Giá trị
SEQ_LEN	5 (5 giao dịch trước)
NUM_FEATURES	15
Input tensor	(1, 5, 15) (thêm chiều batch)
Thiết bị	CPU (torch.device("cpu"))
Padding	Left zero-padding khi chưa đủ 5 giao dịch
File checkpoint	/opt/spark-models/Tham_lstm_attn_checkpoint.pth
Ngưỡng (THRESHOLD)	0.7 (mã nguồn hiện tại) / 0.5 (theo log benchmark gần nhất)
Chuẩn hóa đầu vào	StandardScaler (mean/std từ tập huấn luyện 2018-07-11 → 2018-07-18)

5.3 Nguyên tắc suy diễn không rò rỉ

- **Giao dịch hiện tại không bao giờ nằm trong** `x` của chính nó: dự đoán cho T6 dùng T1→T5; sau khi dự đoán xong, T6 mới được thêm vào lịch sử.
- Vì vậy mô hình luôn dự đoán "tương lai" dựa trên quá khứ, đúng bản chất hệ thống real-time.

6. Nguyên tắc chống rò rỉ dữ liệu (No Data Leakage)

Đây là yếu tố quan trọng nhất để đảm bảo kết quả đánh giá phản ánh đúng năng lực hệ thống trong thực tế. Toàn pipeline tuân thủ nghiêm ngặt các quy tắc sau:

#	Quy tắc	Áp dụng tại
1	TX_FRAUD chỉ là ground truth — chỉ dùng để so sánh với <code>FRAUD_PREDICTION</code> khi đánh giá, không bao giờ dùng để tính đặc trưng, lịch sử hay xác suất.	Producer, Feature Engineering, Customer History, LSTM Inference
2	Đặc trưng tính trước khi cập nhật state — giao dịch hiện tại chỉ được thêm vào <code>customer_state / terminal_state</code> sau khi các đặc trưng của nó đã được tính xong.	Feature Engineering
3	Giao dịch hiện tại không nằm trong <code>x</code> — chuỗi dự đoán luôn dùng 5 giao dịch trước đó ; giao dịch hiện tại chỉ được thêm vào lịch sử sau khi dự đoán xong.	Customer History, LSTM Inference
4	Tách biệt thời gian gốc và thời gian thực — <code>TX_DATETIME</code> (thời điểm giao dịch lịch sử) không được dùng để tính latency; latency real-time chỉ dựa trên <code>PRODUCER_TIMESTAMP</code> (thời điểm gửi thật) và <code>SINK_TIMESTAMP</code> (thời điểm ghi thật).	Producer, PostgreSQL Sink
5	Chuẩn hóa bằng tham số huấn luyện — không fit lại scaler trên dữ liệu real-time (tránh nhìn thấy tương lai); dùng cố định mean/std từ cửa sổ huấn luyện.	LSTM Inference
6	maxOffsetsPerTrigger + sắp xếp theo TX_DATETIME — đảm bảo các giao dịch trong một micro-batch được xử lý đúng thứ tự thời gian như dữ liệu gốc.	Feature Engineering

Việc so sánh cuối cùng được thực hiện trên cùng một `TRANSACTION_ID`:

TX_FRAUD (ground truth, do dataset cung cấp) vs
--

7. Đo lường hiệu năng thời gian thực

7.1 Các mốc thời gian

Mốc	Được tạo bởi	Ý nghĩa
TX_DATETIME	Dataset gốc	Thời điểm giao dịch lịch sử (KHÔNG dùng cho latency)
PRODUCER_TIMESTAMP	Producer	Thời điểm giao dịch thực sự được gửi vào Kafka
PREDICTION_START_TIMESTAMP	LSTM Inference	Thời điểm bắt đầu inference
PREDICTION_END_TIMESTAMP	LSTM Inference	Thời điểm kết thúc inference
SINK_TIMESTAMP	PostgreSQL Sink	Thời điểm giao dịch được ghi xuống PostgreSQL

7.2 Các chỉ số độ trễ

Prediction latency — thời gian mô hình suy diễn (không gồm Kafka/Spark):

```
PREDICTION_LATENCY_MS = PREDICTION_END_TIMESTAMP - PREDICTION_START_TIMESTAMP
                        = (perf_counter_end - perf_counter_start) * 1000
```

End-to-end latency — thời gian trọn vòng từ lúc producer gửi đến lúc ghi vào PostgreSQL (bao gồm Kafka, Spark, Feature Engineering, LSTM, Sink):

```
END_TO_END_LATENCY_MS = unix_micros(SINK_TIMESTAMP) - unix_micros(PRODUCER_TIMESTAMP)
```

7.3 Throughput

Throughput (tx/s) = số giao dịch đã xử lý / thời gian thực tế của pipeline

- Trong `evaluate_realtime.py`, throughput được đo từ khoảng thời gian thực (measurement start → measurement end), **không dùng** `TX_DATETIME`.

7.4 Bộ công cụ đánh giá (`evaluate/evaluate_realtime.py`)

Sau khi pipeline chạy xong, script này kết nối PostgreSQL, đọc toàn bộ bảng `fraud_predictions` và tính:

- Dataset summary:** tổng giao dịch, số gian lận thật/giả định.
- Confusion matrix:** TN, FP, FN, TP.
- Classification metrics:** Accuracy, Precision, Recall, F1, FPR.
- Probability metrics:** ROC-AUC, Average Precision, Precision-Recall curve (lưu PNG).
- Latency statistics:** trung bình, P50, P95, P99, min, max của prediction latency và end-to-end latency.
- Throughput:** giao dịch/giây.
- Precision@Top-K:** Precision@50, @100, @200 theo xác suất dự đoán giảm dần.

Kết quả được lưu vào thư mục `evaluation_results/` (CSV lịch sử + JSON mới nhất + ảnh PR curve). Ngoài ra `export_postgres_to_excel.py` xuất toàn bộ bảng sang file `.xlsx`.

8. Quy trình vận hành

8.1 Reset hệ thống (reset_realtime.bat)

Chạy khi muốn **chạy benchmark mới hoàn toàn sạch** (dữ liệu cũ sẽ bị xóa):

1. Kiểm tra các container `spark`, `kafka`, `postgres` đang chạy.
2. Xóa checkpoint của 4 Spark job (`lstm_inference`, `customer_history`, `feature_engineering`, `postgres_sink`).
3. Xóa dữ liệu features/history trong `/opt/spark-data`.
4. `TRUNCATE TABLE fraud_predictions` trong PostgreSQL (yêu cầu đếm = 0).
5. Xóa 3 topic Kafka (`transactions`, `transactions_features`, `fraud_predictions`) rồi tạo lại sạch (1 partition, replication 1).
6. Xác nhận các topic rỗng và sẵn sàng cho benchmark.

8.2 Khởi động pipeline (start_realtime.bat)

Mở **6 cửa sổ terminal** theo đúng thứ tự hạ nguồn trước, thượng nguồn sau:

Bước	Cửa sổ	Lệnh
1/6	06 - PostgreSQL Sink	<code>spark-submit --packages ... spark-sql-kafka-0-10_2.13:4.0.1,org.postgresql:postgresql:42.7.7 /opt/spark-apps/postgres_sink.py</code>
2/6	05 - LSTM Inference	<code>spark-submit ... /opt/spark-apps/lstm_inference.py</code>
3/6	04 - Customer History	<code>spark-submit ... /opt/spark-apps/customer_history.py</code>
4/6	03 - Feature Engineering	<code>spark-submit ... /opt/spark-apps/feature_engineering.py</code>
5/6	02 - Kafka Producer	<code>python transaction_producer.py --rate 10 --limit 500000 --print-every 1000</code>
6/6	01 - Evaluate CPU/RAM	<code>python monitor_resources.py --duration 300 --interval 1</code>

Mỗi bước cách nhau 5 giây (`timeout /t 5`) để các Spark stream kịp READY trước khi producer bắt đầu gửi.

8.3 Dừng pipeline (stop_realtime.bat)

- Đóng 6 cửa sổ CMD bằng `taskkill` theo tiêu đề cửa sổ.
- Trong container Spark: `pkill -f` các file `postgres_sink.py`, `lstm_inference.py`, `customer_history.py`, `feature_engineering.py`.
- **Không xóa** dữ liệu PostgreSQL, Kafka và checkpoint.

8.4 Quy trình benchmark (so sánh tốc độ)

Thực hiện tuần tự cho từng tốc độ, mỗi vòng đều reset lại từ đầu:

```
1. reset_realtime.bat
2. start_realtime.bat (4 Spark stream READY)
3. Producer: python transaction_producer.py --rate <R> --limit 500000 --print-every 1000
4. Chờ PostgreSQL Sink hoàn tất (END MARKER)
5. python evaluate_realtime.py
6. Xuất Excel: python export_postgres_to_excel.py
```

Với $\langle R \rangle \in \{10, 50, 100, 500\}$ tx/s. Kết quả của từng vòng được lưu thành các file riêng (ví dụ `50tx-s_realtime_evaluation_latest.json`, `100tx-s_...`, `500tx-s_...`).

9. Kết quả benchmark thực tế

Các kết quả dưới đây được trích từ các file đánh giá trong thư mục `exports/10_50_100_500tx-s_evaluate/(*_realtime_evaluation_latest.json)`. Mỗi benchmark xử lý **500.000 giao dịch** với một tốc độ phát khác nhau; pipeline được reset hoàn toàn trước mỗi vòng.

9.1 Bảng tổng hợp so sánh 3 tốc độ

Chỉ số	50 tx/s	100 tx/s	500 tx/s
Tổng giao dịch	500.000	500.000	500.000
Gian lận thật (ground truth)	3.592	3.592	3.592
Dự đoán gian lận	2.429	2.429	2.425

Ma trận nhầm lẫn

	50 tx/s	100 tx/s	500 tx/s
TN	496.252	496.252	496.252
FP	156	156	156
FN	1.319	1.319	1.323
TP	2.273	2.273	2.269

Chỉ số phân loại

Chỉ số	50 tx/s	100 tx/s	500 tx/s
Accuracy	0,99705	0,99705	0,99704
Precision	0,93578	0,93578	0,93567
Recall	0,63280	0,63280	0,63168
F1-score	0,75502	0,75502	0,75420
FPR	0,00031	0,00031	0,00031
ROC-AUC	0,97205	0,97205	0,97199
Average Precision	0,80547	0,80545	0,80573
Precision@50 / @100 / @200	1,0	1,0	1,0

Độ trễ dự đoán (prediction latency)

Chỉ số	50 tx/s	100 tx/s	500 tx/s
Trung bình (ms)	64,02	137,75	5.329,14
P50 (ms)	27	53	4.671
P95 (ms)	205	396	9.884
P99 (ms)	914	2.583	9.884
Min (ms)	3	10	142
Max (ms)	1.099	2.583	9.884

Độ trễ end-to-end (từ Producer → PostgreSQL Sink)

Chỉ số	50 tx/s	100 tx/s	500 tx/s
Trung bình (ms)	16.373	22.058	416.443
P50 (ms)	14.755	19.332	372.603
P95 (ms)	23.385	30.320	625.584
P99 (ms)	54.535	110.375	649.157
Max (ms)	137.565	140.489	659.551

Throughput thực tế

Chỉ số	50 tx/s	100 tx/s	500 tx/s
Số giao dịch	500.000	500.000	500.000
Thời lượng pipeline (giây)	10.012,8	5.014,4	1.619,9
Throughput thực (tx/s)	49,94	99,71	308,66

9.2 Phân tích kết quả

- Chất lượng dự đoán không đổi theo tốc độ:** các chỉ số Accuracy/Precision/Recall/F1/ROC-AUC gần như giống hệt nhau ở cả 3 mức tốc độ. Điều này cho thấy **tốc độ phát không ảnh hưởng đến độ chính xác** của mô hình — độ trễ chỉ ảnh hưởng đến *thời điểm* có kết quả, không ảnh hưởng đến *nội dung* dự đoán.
- Độ chính xác tổng thể rất cao (Accuracy ≈ 99,7%)** với **Precision ≈ 93,6%** — nghĩa là khi hệ thống báo gian lận, ~93,6% trường hợp là đúng. **Recall ≈ 63,2%** cho thấy hệ thống bắt được khoảng 2/3 số vụ gian lận thật — mức hợp lý cho bài toán phát hiện gian lận trong thời gian thực với ngưỡng cố định.
- Latency tăng nhanh theo tốc độ:** - Ở **50 tx/s**: prediction latency trung bình chỉ **~64 ms**, end-to-end trung bình **~16 giây** — hoàn toàn trong vùng chấp nhận được. - Ở **100 tx/s**: prediction latency **~138 ms**, end-to-end **~22 giây** — vẫn ổn. - Ở **500 tx/s**: prediction latency vọt lên **~5,3 giây**, end-to-end trung bình **~416 giây (~7 phút)** — pipeline **không theo kịp tốc độ phát**; dữ liệu bị ùn ứ trong Kafka.
- Throughput thực tế ở 500 tx/s chỉ đạt ~308,66 tx/s** (thấp hơn mục tiêu 500 tx/s ~38%). Đây chính là nguyên nhân độ trễ tăng vọt: hệ thống xử lý chậm hơn tốc độ dữ liệu đến nên hàng đợi Kafka dài dần.
- Precision@Top-K = 1,0** ở mọi tốc độ: khi sắp xếp 500.000 giao dịch theo xác suất gian lận giảm dần, toàn bộ 200 giao dịch đứng đầu đều là gian lận thật — điểm rất mạnh nếu muốn sử dụng ngưỡng động (dynamic threshold) hoặc xếp hạng rủi ro.

10. Mức tiêu thụ tài nguyên hệ thống

Dữ liệu được ghi lại bởi `monitor_resources.py` (file `resource_metrics.csv`). Ví dụ một mẫu đo trong quá trình chạy:

Container	CPU	Memory
spark	~383%	~1,34 GiB / 7,68 GiB (17,4%)
kafka	~7% – 35%	~1,13 GiB / 7,68 GiB (14,7%)
postgres	~0%	~46 MiB / 7,68 GiB (0,6%)

Nhận xét:

- **Spark là thành phần tốn tài nguyên nhất** (hàng trăm % CPU do chạy 4 streaming job đồng thời, bao gồm cả suy diễn PyTorch) — đây là điểm nghẽn chính khi tăng tốc độ giao dịch.
- **Kafka** tiêu thụ vừa phải (~1,1 GiB RAM, CPU dao động theo lưu lượng).
- **PostgreSQL** rất nhẹ ở mức lưu lượng này.

11. Kết luận

11.1 Tóm tắt quá trình real-time

1. **Producer** phát lại 500.000 giao dịch lịch sử vào Kafka theo tốc độ cấu hình (10/50/100/500 tx/s), gắn `PRODUCER_TIMESTAMP` làm mốc thời gian thực.
2. **Feature Engineering** tính 15 đặc trưng (thời điểm, cửa sổ khách hàng/terminal 1-7-30 ngày) theo nguyên tắc không rò rỉ.
3. **Customer History** và **LSTM Inference** xây chuỗi 5 giao dịch trước của từng khách hàng, chuẩn hóa theo scaler huấn luyện và dự đoán xác suất gian lận bằng mô hình LSTM + Attention.
4. **PostgreSQL Sink** ghi kết quả kèm `SINK_TIMESTAMP`, cho phép tính độ trễ end-to-end chính xác.
5. **Bộ đánh giá** so sánh `FRAUD_PREDICTION` với ground truth `TX_FRAUD` và đo latency/throughput.

11.2 Kết quả đạt được

- **Chất lượng dự đoán ổn định:** Accuracy $\approx$ 99,7%, Precision $\approx$ 93,6%, ROC-AUC $\approx$ 0,972 ở mọi tốc độ.
- **Độ trễ thấp ở tốc độ thấp:** end-to-end ~16 giây (50 tx/s), ~22 giây (100 tx/s).
- **Hệ thống có năng lực xử lý ~300 tx/s:** ở 500 tx/s pipeline bắt đầu quá tải (end-to-end ~7 phút, throughput thực chỉ ~309 tx/s).

11.3 Hạn chế và hướng cải tiến

1. **Nghẽn cổ chai ở 500 tx/s:** cần tối ưu Feature Engineering (giảm overhead Python trong `foreachBatch`), tăng số partition Kafka để song song hóa, hoặc tăng tài nguyên cho container Spark.
2. **Trạng thái trong bộ nhớ driver:** `defaultdict` / `deque` không được checkpoint — khi job khởi động lại giữa chừng, lịch sử sẽ được dựng lại từ đầu. Có thể dùng Spark state store (`mapGroupsWithState`) để lưu trạng thái bền vững.

3. **Ngưỡng cố định:** Recall ~63% có thể được cải thiện bằng ngưỡng động hoặc tối ưu threshold theo chi phí FP/FN.
4. **Prediction latency gắn với batch:** mọi giao dịch trong một micro-batch nhận cùng latency của batch; có thể giảm bằng cách giảm kích thước batch hoặc dùng streaming inference.

11.4 Các file liên quan

File	Vai trò
producer/transaction_producer.py	Producer phát lại giao dịch
spark/feature_engineering.py	Tính 15 đặc trưng
spark/customer_history.py	Xây chuỗi 5 giao dịch trước
spark/lstm_inference.py	Suy diễn LSTM + Attention
spark/postgres_sink.py	Ghi kết quả vào PostgreSQL
spark/model_inference.py, spark/test_lstm.py	Định nghĩa kiến trúc mô hình & kiểm thử
evaluate/evaluate_realtime.py	Đánh giá toàn diện
evaluate/monitor_resources.py	Giám sát CPU/RAM
evaluate/export_postgres_to_excel.py	Xuất dữ liệu sang Excel
docker-compose.yml, Dockerfile	Hạ tầng
start_realtime.bat, stop_realtime.bat, reset_realtime.bat	Vận hành
README_realtime.txt, evaluate/README_evaluation.txt	Hướng dẫn

Hết báo cáo.